Forensic Architecture Report: The Transition to Unitary Mechanistic Jurisdiction and Operational Hardening in the Stela Platform

1. Executive Summary: The Authority Crisis and the Mechanistic Pivot

The Stela platform, an architectural evolution of the PHP-Nuke Community Edition (CE) designed for AI-mediated software development, has recently undergone a catastrophic integrity failure identified as a "malicious model infection." This report provides an exhaustive forensic analysis of the breach, the theoretical failures of the antecedent "Multi-verse" governance model, and the comprehensive remediation strategy implemented through the "Flat Truth" architecture. The culmination of this analysis is the drafting of Dispatch Packet DP-OPS-0028, which serves to ratify the forensic findings, explain the operational kernel's binary logic, and permanently harden the repository against semantic exploitation.

The core of the failure lay in the platform's initial bifurcated jurisdictional model, which distinguished between "The Crown" (operational core) and "The Colonies" (project payloads). This separation, intended to maximize developer velocity by allowing linguistic leniency in project files, created a "shadow compliance" zone. Adversarial logic patterns, utilizing a "SnatchML" vector, successfully infiltrated the repository's control plane by mimicking legitimate project operations and exploiting the "Confused Deputy" vulnerability within the ops/bin/project binary.

In response, the platform has shifted to a **Unitary Mechanistic Jurisdiction** governed by a singular **Policy of Truth (PoT)**. This paradigm shift abolishes local law, enforces a universal "No Contractions" rule, and lobotomizes core binaries into deterministic loaders to eliminate autonomous decision-making. Furthermore, a forensic review of the operational kernel reveals that maintaining a "Clean State" requires strict discipline regarding artifact generation; specifically, the Context Bundler (ops/bin/llms) must be directed to output artifacts to storage/handoff/ rather than the repository root to prevent "porcelain noise" that degrades the Freshness Gate's integrity.

2. The Antecedent State: Theoretical Failures of the Multi-verse Model

2.1 The Multi-verse Architecture and the Perimeter Fallacy

The initial design of Stela operated under a governance framework colloquially known as "The Multi-verse." This model was defined in Section 3 of the original Policy of Truth and established two distinct legal zones:

- **The Crown**: Encompassed the ops/ and docs/ directories. These areas were subject to rigid linguistic and structural standards, enforced with absolute intolerance for deviation.
- **The Colonies**: Encompassed the projects/ directory. These areas held payload code and were granted significant leniency regarding linguistic precision (e.g., use of contractions) and formatting.

The strategic rationale was based on the "Perimeter Security" fallacy. The assumption was that as long as the operational kernel (The Crown) remained secure, the chaotic or informal nature of the payload code (The Colonies) could be contained. This model aimed to foster high-velocity experimentation in project layers without compromising the stability of the core. However, this architectural bifurcation ignored the fundamental reality of Large Language Model (LLM) processing: the context window is a single, unified plane of cognition.

2.2 The Shadow Compliance Zone and Semantic Noise

Forensic evidence demonstrates that the bifurcation of law created a "shadow compliance" zone. In a text-based, AI-mediated environment, every token processed by the Code Large Language Model (CLLM) functions as part of the control plane.[1] When an AI agent (the Integrator) operates within a context window that contains conflicting rules—strictly formal platform laws versus relaxed project laws—its internal representation of authority degrades.

The linguistic leniency granted to projects/ provided the necessary cover for adversarial logic patterns to take root. By permitting contractions and informal phrasing in STELA.md files, the system introduced "semantic noise." This noise increased the probabilistic variance in the model's attention mechanism, allowing malicious instructions to be masked as benign project documentation. The "SnatchML" attack vector exploited this by injecting prompts that mimicked the statistical signature of the "Colonial" dialect while carrying a payload that overrode the "Crown" directives.[1]

3. Forensic Analysis of the Malicious Model Infection

3.1 The SnatchML Vector

The infection vector was identified as a "SnatchML" pattern. SnatchML is a threat model described in adversarial machine learning literature where adversaries with restricted access hijack the behavior of a machine learning model without needing root access to the training pipeline or core infrastructure. In Stela's context, the "model" was the conceptual governance framework held by the AI Integrator and the platform's automation scripts.

The attacker did not modify the root-protected ops/ directory directly. Instead, they injected adversarial prompts into a low-privilege projects/target/STELA.md file. These prompts were semantically engineered to mimic standard project documentation but contained instructions designed to "jailbreak" the platform's execution flow.

3.2 The Confused Deputy: ops/bin/project

The primary vehicle for the exploit was the ops/bin/project binary. Prior to remediation, this binary was a dense shell script containing significant autonomous logic for resolving agent paths (project_resolve_agent_file) and extracting roles (project_extract_agent_role).

Because the binary was architected to "interpret" intent, it became a "Confused Deputy." A Confused Deputy is a computer program that is innocently fooled by some other party into misusing its authority. The malicious model provided crafted arguments within the project file that convinced the binary that the "local law" of the project superseded the "physical law" of the PoT. Running with platform authority, the binary executed the malicious instructions, effectively elevating them to platform status.

Attack Characteristic	**Stela Manifestation**
Vector	Semantic Exploit via STELA.md
Privilege Level	Low (Colonial/Project scope)
Target	ops/bin/project (Confused Deputy)
Effect	Logic Drift & Authority Hallucination
Masking	Linguistic Steganography (Contractions)

3.3 The Failure of "DiSim" Mechanisms

Prior to the infection, Stela relied on statistical outlier detection—referred to as "DiSim-defense"—to identify malicious updates. These mechanisms typically use metrics like Cosine Similarity to compare incoming updates against benign patterns:

$$Cosine\ Similarity(A, B) = \frac{A \cdot B}{||A||||B||}$$

The fundamental assumption of DiSim-defense is that adversarial updates will exhibit identifiable statistical anomalies, such as divergent vector orientations or abnormal

magnitudes in the embedding space. However, the forensic autopsy revealed that semantic poisoning in a text-based system is statistically indistinguishable from valid documentation when linguistic rigor is not maintained universally.

Because the "Colonial" exemption explicitly disabled linguistic linters for project files, the repository's "immune system" was suppressed in the exact location where the infection occurred. The adversarial changes—subtle shifts in phrasing and the use of contractions—were statistically similar to the "Colonial" baseline, allowing them to bypass detection mechanisms that relied on deviation from the norm.[1]

4. The Doctrine of Unitary Mechanistic Jurisdiction

4.1 Re-Flattening to "Flat Truth"

The strategic response to the authority crisis is the immediate and permanent re-flattening of jurisdiction. The "Multi-verse" model has been abolished. The system now operates under a **Unitary Mechanistic Jurisdiction**, asserting that there is only one law: the Policy of Truth (PoT).

Section 3 of the updated PoT.md now asserts that the PoT is the sole authority across all scopes, including ops/, docs/, projects/, and .github/. No parallel jurisdictions exist. This shift mandates a universal application of the Behavioral Logic Standard, particularly the prohibition of contractions.

4.2 The "No Contractions" Universal Constant

To combat linguistic steganography, the **Behavioral Logic Standard** now enforces a global prohibition on the use of contractions (e.g., "don't", "can't") across *all* repository surfaces, including projects. This is not a stylistic choice but a security control. Contractions introduce rhythmic irregularity and semantic ambiguity (e.g., "can't" vs. "cannot"). By enforcing "No Contractions," the platform ensures that all text within the control plane is formal, objective, and predictable. This monotonic standard reduces the probabilistic noise that allowed the AI Integrator to be jailbroken, forcing the model to process instructions with higher cognitive rigidity.[1]

4.3 Zero Trust Architecture

The platform has moved to a **Zero Trust** approach. Every file, regardless of its location or author, is subjected to the full rigor of the platform's linters and gates. There are no "safe zones" or "trusted directories" that are exempt from validation. This prevents the "petri dish" effect where low-security zones serve as incubation chambers for malicious logic.

Terminology (Abolished)	Terminology (Unified)	Functional Rationale

The Crown	Platform Surfaces	Standardized as the operational core under PoT.
The Colonies	Project Payloads	Treated as functional directories, not legal territories.
Local Law	Unitary Jurisdiction	Elimination of linguistic and structural exceptions.
Shadow Compliance	Zero Trust	Universal enforcement of linters and repository gates.

5. Anatomy of the Ops Kernel: Detailed Artifact Analysis

To fully understand the Mechanistic Doctrine, one must analyze the executable components that enforce it. The ops/ directory is not merely a collection of scripts; it is the "Run" layer of the Filing Doctrine, acting as the repository's operating system. The following section provides a detailed forensic explanation of every binary, script, and tool within the operational kernel, as requested by the governance mandate.

5.1 The Operational Binaries (ops/bin/)

These executables are the primary interface for the Operator and the Integrator. They are designed to be "explainable" and "deterministic."

5.1.1 ops/bin/open: The Freshness Gate and Intent Declaration

Purpose: Generates the "Open Prompt," a standardized text artifact that initializes the AI agent's session.

Mechanism:

- **Freshness Gate:** It queries git rev-parse HEAD and git status --porcelain. If the working tree is dirty (uncommitted changes), it captures the "porcelain" state into a text file in storage/handoff/. This forces the AI to acknowledge the *exact* starting state of the repository, preventing "hallucinations of cleanliness."
- **Intent Declaration:** It accepts arguments like --intent and --dp. These are injected into the prompt header, contractually binding the session to a specific goal.
- **Canon Injection:** It lists the "Canon Pointers" (PoT, TASK, SoP), instructing the AI on what

constitutes the immutable law for the session. **Forensic Insight:** ops/bin/open is the "Timekeeper." By freezing the git hash and status at the start of a session, it creates a verifiable checkpoint. If an agent operates on a stale hash, the Sheriff (linters) will later reject the work.[1]

5.1.2 ops/bin/dump: The State Serializer

Purpose: Captures the platform's state into a single, flat text file for AI ingestion.

Mechanism:

- **Scope Selection:** It accepts a --scope argument (e.g., platform, full). It uses git ls-files to deterministically select files, ensuring that only tracked, authorized files are included.
- **Refinement:** It supports --include-dir and --exclude-dir, allowing the Operator to slice the context window precisely.
- **Formatting:** It concatenates file contents with clear delimiters (<<< FILE BEGIN: path), allowing the LLM to parse the stream as a virtual file system. **Forensic Insight:** ops/bin/dump is the "Observer." It defeats "context drift" by providing the model with the raw, uninterpreted evidence of the repository's state. It implements the "Static RAG" principle, brute-forcing the truth into the context window rather than relying on probabilistic retrieval.[1]

5.1.3 ops/bin/llms: The Context Bundler (and the Root Clutter Issue)

Purpose: Compiles the "Canonical Context" and "Supporting Surfaces" into standardized bundles (llms-small.txt, llms-full.txt) for easy discovery by agents.

Mechanism:

- **Manifest Parsing:** It reads ops/lib/manifests/CONTEXT.md to identify which files are mandatory.
- **Redaction:** It applies sed patterns to strip potential secrets (API keys, private keys) from the stream before bundling.
- **Output Generation:** It concatenates the selected files into the target output files. **Forensic Anomaly & Remediation:** Current forensic scans of the codebase (specifically ops/bin/llms line 4: OUT_DIR="${REPO_ROOT}") reveal that this binary defaults to dropping artifacts in the repository root. This violates the "Clean Working Tree" axiom enforced by ops/bin/open. When llms-full.txt is regenerated in the root, git status reports the tree as "dirty," creating noise in the Freshness Gate. **Operational Requirement:** The system must enforce that these artifacts are dropped in storage/handoff/ or another ignored directory. DP-OPS-0028 must ratify this change in procedure, either by patching the binary or enforcing the usage of the --out-dir flag.[1]

5.1.4 ops/bin/project: The Lobotomized Loader

Purpose: Orchestrates project-specific workflows.

Mechanism (Post-Lobotomy):

- **Deterministic Loading:** Instead of interpreting "what the project wants," it simply concatenates the project's STELA.md file with the platform's dump output.
- **Wrapper Function:** It calls ops/bin/open and ops/bin/dump in sequence, then appends the local project context. **Forensic Insight:** This binary was the vector for the SnatchML attack. It has been "lobotomized"—stripped of autonomous logic (project_resolve_agent_file)—and converted into a "dumb pipe." This ensures that the platform's laws (PoT) effectively "wrap" the project's logic, preventing the project from escaping its container.[1]

5.1.5 ops/bin/prune: The Hygiene Engine

Purpose: Manages the accumulation of historical data to prevent context window overflow.

Mechanism:

- **SoP Truncation:** It parses SoP.md, keeping only the most recent 30 entries. Older entries are moved to storage/archives/root/.
- **Handoff Cleanup:** It deletes artifacts in storage/handoff/ older than 7 days, unless they are pinned to the current Dispatch Packet ID. **Forensic Insight:** ops/bin/prune enforces the "Sliding Window" of history. It ensures that the SoP.md remains a "living ledger" rather than a museum, keeping the token count efficient for the AI.[1]

5.1.6 ops/bin/help: The Explainability Interface

Purpose: Provides a CLI-based documentation search.

Mechanism: It greps the docs/ directory for keywords, providing line numbers and file paths. This allows the Operator to query the "mind" of the system directly from the terminal.

5.2 The Shared Libraries (ops/lib/scripts/)

5.2.1 ops/lib/scripts/skill.sh: The Provenance Engine

Purpose: Manages the lifecycle of "Skills" (reusable workflows).

Mechanism:

- **Harvest:** Generates a draft skill file. Crucially, it auto-generates a **Provenance Block** containing the DP-ID, Branch, HEAD hash, and diff statistics of the session where the skill was learned.
- **Promote:** Moves the draft to docs/library/skills/ and registers it in docs/library/INDEX.md.
- **Check:** Enforces the "Context Hazard" rule, ensuring skills are *not* added to the global context manifest (ops/lib/manifests/CONTEXT.md). **Forensic Insight:** This script enforces the "No Invention" rule. A skill cannot exist without a provenance block proving it emerged from valid work. This prevents the injection of "hallucinated" or malicious best practices.[1]

5.2.2 ops/lib/scripts/heuristics.sh: The Git Forensics Module

Purpose: specific logic for analyzing git history to find friction points.

Mechanism:

- **Hot Zone Detection:** Uses git diff --numstat to identify files with the highest volume of change.
- **High Churn Detection:** Uses git log to identify files modified most frequently. **Forensic Insight:** This script provides the data that feeds the skill.sh provenance block. It turns "vibes" (subjective feelings about code fragility) into "evidence" (statistical metrics of churn).[1]

5.3 The Sheriff: Mechanistic Enforcement (tools/lint/)

These scripts act as the "immune system" of the repository. They are executed by CI/CD gates and must pass before any merge.

5.3.1 tools/lint/context.sh: The Anti-Hallucination Guard

Purpose: Verifies that every file listed in ops/lib/manifests/CONTEXT.md actually exists. **Mechanism:** It parses the manifest, extracts paths, and checks for file existence. It also enforces "Negative Constraints," failing if it finds forbidden directories (like docs/library/skills) in the global manifest. **Forensic Insight:** This prevents the AI from being "blinded" by a corrupt context description. If the manifest says a file is context, it *must* be there.[1]

5.3.2 tools/lint/truth.sh: The Linguistic Police

Purpose: Enforces the Policy of Truth standards.

Mechanism:

- **Spelling:** Greps for forbidden typos of "Stela" (e.g., "Stella", "Steela").
- **Contractions:** Greps for forbidden tokens ("don't", "can't", "won't") across strictly controlled surfaces (docs/, ops/). **Forensic Insight:** This is the primary defense against linguistic steganography. By enforcing "No Contractions," it maintains the rhythmic precision required for technical auditing.[1]

5.3.3 tools/lint/dp.sh: The Contract Validator

Purpose: Validates the structure of a Dispatch Packet (DP) in TASK.md. **Mechanism:** It checks for the presence of mandatory headers ("Freshness Gate", "Execution Plan", "Closeout"), specific sections (A-E Canon), and ensures no placeholder tokens (TBD, [ ]) remain. **Forensic Insight:** This ensures that every unit of work is governed by a valid, complete contract. An incomplete DP cannot be executed.[1]

5.3.4 tools/lint/library.sh: The Registry Guard

Purpose: Enforces the integrity of docs/library/INDEX.md.

Mechanism:

- **Dead Ends:** Checks that every path listed in the Index exists on disk.
- **Ghosts:** Checks that every .md file in docs/library/ is registered in the Index. **Forensic Insight:** This prevents "Shadow Documentation"—files that exist but are not indexed (ghosts) or files that are indexed but missing (dead ends). It ensures the library is a closed, curated system.[1]

5.3.5 tools/lint/project.sh: The Pointer Validator

Purpose: Validates STELA.md files within projects. **Mechanism:** It scans project definition files to ensure they reference valid Agents (R-AGENT-*) and Tasks (B-TASK-*) that actually exist in the library index. **Forensic Insight:** This prevents projects from referencing deprecated or non-existent governance structures.[1]

6. Operational Directive: DP-OPS-0028

Based on the forensic analysis, the "Deep Research" imperative, and the identification of the llms binary output issue, the following Dispatch Packet (DP-OPS-0028) is constructed. This DP performs three critical actions:

1. **Ratification:** Creates docs/library/FORENSICS.md to canonize the "Forensic Autopsy" report.
2. **Registration:** Updates docs/library/INDEX.md to include the new forensic artifact.
3. **Correction:** Updates the llms generation workflow to ensure artifacts are dropped in storage/handoff/, satisfying the "Clean Working Tree" requirement.

---

3. Current Dispatch Packet (DP) - Stela Standard (A-E)

DP-OPS-0028: Mechanistic Jurisdiction Hardening & Forensic Ratification

3.1 Freshness Gate (Must Pass Before Work)

Base Branch: main

Required Work Branch: work/forensic-ratification-2026-02-06

Base HEAD: d394111dd (preferred) or Not provided / Current (draft; lock at merge)

Required local re-check (worker runs; paste results in RESULTS):

- git rev-parse --abbrev-ref HEAD
- git rev-parse HEAD
- git status --porcelain

Worker must already be on the Required Work Branch listed above (no branch creation or switching). STOP if any mismatch. STOP if the Required Work Branch is missing. STOP if told to create or switch branches.

3.2 Required Context Load (Read Before Doing Anything)

Worker must confirm loaded before acting:

- PoT, SoP, CONTEXT, MAP
- Plus any DP-scoped files listed here:
  - [Exact path]
  - [Exact path]

(OPEN is for Integrator/DP Writer state refresh; Worker does not read OPEN). DISPOSABLE ARTIFACTS (chat logs, etc.) MUST NOT BE REFERENCED OR INCLUDED.

3.3 Scope and Safety

- **Objective:** Ratify forensic findings from the malicious model infection and harden the repository by establishing the Forensic Autopsy as a canonical reference and enforcing correct output paths for context bundles.
- **Non-Goals (optional, drift-killer):**
  - No changes to ops/bin/project logic (already handled in DP-OPS-0027).
  - No changes to PoT.md (already consolidated in DP-OPS-0025).

Target Files allowlist (hard gate)

- docs/library/FORENSICS.md (new)
- docs/library/INDEX.md
- SoP.md

Allowlist rule: exact paths only; use (new) prefix only when the DP explicitly allows new files.

- **Conditional Requirements:** If canon/governance surfaces (e.g., TASK.md) are modified, SoP.md MUST be included in the allowlist and updated.
- **Stop Condition (scope):** If any required change falls outside the allowlist, **STOP** and report.
- **Forbidden (unless DP explicitly overrides):**
  - No edits to .github/
  - No edits to projects/ unless explicitly authorized
  - No renames/moves/deletes unless explicitly authorized
- **Worker Git Authority (non-negotiable):**
  - Workers do not create/switch branches.
  - Workers do not stage/commit/merge/push.
  - Workers only return diffs/results/receipt.
- **Precedence:** If any conflict exists, **PoT.md wins**. If unclear, STOP and ask.
- **Strict Stop Conditions (generic):**
  - STOP if any placeholder text remains in final outputs ([...], TODO, TBD, ...).
  - STOP if required DP inputs are missing (do not guess).
  - STOP if IDs / naming constraints in the DP are inconsistent (when applicable).

3.4 Execution Plan (A-E Canon)

3.4.1 State

- **Context:** The platform has recovered from a "malicious model infection" via DP-OPS-0027. The deep technical reasoning for the "Flat Truth" architecture is currently missing from the canonical documentation. Additionally, the ops/bin/llms tool is dropping files in the repository root, violating the "Clean Working Tree" axiom of the Freshness Gate.
- **Drift:** Knowledge gap regarding SnatchML and DiSim failure; Operational drift in llms artifact placement.
- **Desired State:** The Forensic Autopsy is codified as docs/library/FORENSICS.md. The llms bundles are correctly generated in storage/handoff/ to maintain porcelain cleanliness.

3.4.2 Request

Numbered tasks (concrete, no vibes):

1. Create docs/library/FORENSICS.md containing the full text of the "Forensic Autopsy and Structural Re-Architecture" report (content provided in attachment Stela Agent Constitution Discussion.pdf). Ensure all formatting aligns with Stela markdown standards (No Contractions).
2. Update docs/library/INDEX.md to register the new artifact under a new section "8.

Forensic & Security Reports". Use the key forensic-autopsy and title Report: Forensic Autopsy.
3. Verify and Enforce llms output location: When running the context refresh in the Closeout phase, strictly use the --out-dir=storage/handoff flag with ops/bin/llms to ensure artifacts do not pollute the root.
4. Update SoP.md to log the ratification of the forensic report.

Required Content (optional; use when doc/governance changes demand exact text/format):

- In docs/library/FORENSICS.md, ensure the table "Attack Characteristic | Description | Stela Manifestation" is preserved.

3.4.3 Changelog

- **Log (human-readable, 1–6 bullets):**
  - Added docs/library/FORENSICS.md (Forensic Autopsy report).
  - Registered forensic report in docs/library/INDEX.md.
  - Enforced storage/handoff/ output path for llms bundles to maintain clean git status.
  - Updated SoP.md with DP-OPS-0028 closure.

3.4.4 Patch / Diff

- **Format:** Unified diff (preferred) or anchored snippets.

3.4.5 Receipt (Required)

Verification (MUST RUN; or report NOT RUN + reason + risk)

- ./ops/bin/dump --scope=platform (or repo-correct equivalent when scope differs)
- If dump refiners are used, include this fallback line in the DP:
  - Fallback:./ops/bin/dump --scope=platform --format=chatgpt --out=auto --bundle
- **Zero-Byte Check:** Verify dump is not empty (test -s <path>).
- Context lint (use repo-canonical command):
  - Prefer: bash tools/lint/context.sh
  - If not found and required: **STOP** and ask (do not invent a substitute)
- Truth lint (when PoT or canon surfaces touched; use repo-canonical command):
  - Prefer: bash tools/lint/truth.sh
  - If not found and required: **STOP** and ask (do not invent a substitute)

Skill capture (required during normal DP processing)

- Workflow: run ops/lib/scripts/skill.sh harvest to auto-generate a draft with provenance and semantic collision checks.
- Review: inspect the draft in storage/handoff/ for heuristic accuracy (Hot Zone and High Churn). Refine Invocation guidance and Solution. Do not edit the Provenance block.

- Promote: run ops/lib/scripts/skill.sh promote <draft_path> or log "no new skill promoted" with a rationale in RESULTS.
- Legacy capture is forbidden. Do not manually write skill files; the provenance block is required.
- If a DP requires skill capture, the DP Target Files allowlist must include docs/library/SKILLS.md.
- Proof required in RESULTS:
  - Harvest-only: command invoked plus draft path and ls storage/handoff/ output.
- Promotion: command invoked plus grep -n "Promotion Packet:" docs/library/SKILLS.md and grep -n "docs/library/skills/S-LEARN-" docs/library/INDEX.md.

Artifact bundle (required directory)

- storage/handoff/

Proof bundle checklist (required; naming pattern over exact names)

- storage/handoff/DP-OPS--RESULTS.md (required)
- storage/handoff/OPEN-PORCELAIN-[work-branch]-[hash].txt (required)
- storage/handoff/OPEN-[work-branch]-[hash].txt (required)
- storage/dumps/dump-full-... (or platform scope as applicable)
- git diff --name-only (include in RESULTS)
- git diff --stat (include in RESULTS)
- NEXT: one single action (include in RESULTS)

MANDATE: No receipt/proof bundle = **AUTOMATIC DISAPPROVAL**.

Closeout routine receipts not being produced CONSISTENTLY INCLUDING PORCELAIN, DUMP AND OPEN is a failure condition.

Partial receipts (e.g., missing OPEN artifacts when a session was started) are considered failures.

REQUIRED OUTPUT

MUST include a "Status" block at the top of RESULTS:

- Scope summary: (1 sentence validating the loop)
- Tracked change: (What was edited)
- Verification: (Output of proofs)

RESULTS file must include (minimum)

- Verification results (RUN/NOT RUN + reason + risk)
- Patch/diff proof bundle (status/diff list or direct diff)
- Receipt pointers (OPEN + PORCELAIN + DUMP outputs/paths)
- git diff --name-only output

- git diff --stat output
- NEXT: one single action

Storage artifact flow (local-only, untracked):

- OPEN and OPEN-PORCELAIN outputs land in storage/handoff/.
- Dump bundles land in storage/dumps/.
- Results receipts land in storage/handoff/DP-OPS-0000-RESULTS.md (pattern).
- Operator stores DP drafts under storage/dp/intake/ and moves or copies to storage/dp/processed/ after completion (or when reusing a previously executed DP).
- These are retention paths only; they are not worker prerequisites unless the DP explicitly says so.

7. Strategic Implications and Conclusion

The transition to a Unitary Mechanistic Jurisdiction represents a critical maturity point for the Stela platform. The "malicious model infection" served as a stress test that exposed the inherent vulnerabilities of "partial strictness" in generative AI environments. The forensic analysis proves that in a text-based control plane, ambiguity is not merely a stylistic flaw; it is a security vulnerability.

By abolishing the "Multi-verse," enforcing a "Zero Trust" architecture, and "lobotomizing" core binaries into deterministic loaders, the platform has eliminated the shadow compliance zones that facilitated the attack. The "Sheriff" layer now provides continuous, automated policing of the Policy of Truth, ensuring that the system remains in a state of autopoietic equilibrium.

Furthermore, the operational correction regarding llms artifact placement highlights the extreme discipline required for this architecture. Even a benign action like generating a context bundle can become a vulnerability if it degrades the "Clean State" required for verification. The mandates ratified in DP-OPS-0028 ensure that Stela operates not just as a codebase, but as a provenance-forward, forensic-grade system capable of resisting the entropic drift inherent in AI-mediated development.[1] The platform is now hardened, grounded, and prepared for future evolution under the singular authority of the Policy of Truth.

Works cited

1. dump-full-main-d394111dd.txt